



UNIVERSIDADE POTIGUAR

Projeto Semestral de Avaliação A3

Unidade Curricular

Teoria da computação e compiladores

Compilador minilang

Componentes do Grupo

Manuel Geraldo Fagundes de Oliveira bezerra - 12825215961

Natal - RN
2026.1



Professor Orientador

FELIPE OLIVEIRA DOS SANTOS

SAMARA SOARES LEAL

Natal - RN
2026.2

Compilador Educacional Simplificado

MiniLang — Relatório Técnico

1. Introdução

Compiladores são programas que traduzem código escrito em uma linguagem de alto nível para uma representação executável pela máquina. Eles são um dos sistemas mais complexos e fundamentais da ciência da computação, e seu funcionamento está diretamente enraizado na Teoria da Computação — área que estuda os limites e capacidades dos modelos de computação.

Todo compilador moderno, desde os utilizados em linguagens como Python, Java e C++, segue um pipeline de etapas bem definidas: análise léxica, análise sintática, análise semântica, geração de código intermediário e execução. Cada uma dessas etapas possui um modelo teórico correspondente, estudado formalmente por linguistas e matemáticos antes mesmo de existirem computadores eletrônicos.

O presente trabalho descreve o desenvolvimento do compilador MiniLang, uma linguagem educacional simplificada criada com o objetivo de demonstrar na prática como esses conceitos teóricos se manifestam em um sistema real. A MiniLang foi implementada em Python, com cada etapa do compilador organizada em um módulo separado, tornando explícita a correspondência entre teoria e implementação.

Ao construir o compilador, foi possível observar como um Autômato Finito Determinístico (AFD) reconhece padrões léxicos, como um Autômato de Pilha (PDA) valida estruturas gramaticais, como o conceito de Linguagem Sensível ao Contexto (CSL) se aplica à verificação semântica, e como uma Máquina de Turing serve de modelo para a execução sequencial de instruções.

2. A Linguagem MiniLang

A MiniLang é uma linguagem de programação minimalista com a seguinte sintaxe obrigatória:

```
x = 10
y = x + 5
print(y)
```

Regras da linguagem:

- Uma instrução por linha
- Variáveis representadas por identificadores simples
- Números inteiros
- Operadores de adição (+) e subtração (-)
- Comando print() para exibir valores na saída

3. A Hierarquia de Chomsky e a MiniLang

A Hierarquia de Chomsky, proposta pelo linguista Noam Chomsky em 1956, classifica as linguagens formais em quatro tipos, cada um associado a um tipo de gramática e a um modelo de autômato capaz de reconhecê-las. Essa hierarquia é fundamental para a teoria dos compiladores, pois cada etapa do processo de compilação corresponde a um nível diferente dessa hierarquia.

Tipo	Gramática	Autômato	Exemplo
Tipo 0	Irrestrita	Máquina de Turing	Linguagens recursivamente enumeráveis
Tipo 1	Sensível ao Contexto	Autômato Linearmente Limitado	Análise Semântica da MiniLang
Tipo 2	Livre de Contexto	Autômato de Pilha (PDA)	Análise Sintática da MiniLang
Tipo 3	Regular	Autômato Finito (AFD)	Análise Léxica da MiniLang

A hierarquia é inclusiva: toda linguagem regular é também livre de contexto, que é também sensível ao contexto, que é também reconhecível por uma Máquina de Turing. O que diferencia os níveis é o poder computacional necessário para reconhecê-las.

3.1 Tipo 3 — Linguagens Regulares e o Léxico

As linguagens regulares são o nível mais simples da hierarquia. Elas podem ser descritas por expressões regulares e reconhecidas por Autômatos Finitos Determinísticos (AFD). Na MiniLang, a análise léxica opera nesse nível: identificar se uma sequência de caracteres forma um identificador, um número ou um operador é uma tarefa regular, sem necessidade de memória adicional.

O AFD implementado no arquivo afd.py possui estados explícitos (INICIO, LENDO_ID, LENDO_NUM, LENDO_OPERADOR, LENDO_PAREN, FIM, ERRO) e uma tabela de transição representada como dicionário Python. Para cada estado e categoria de caractere, existe exatamente uma transição possível — característica essencial de um autômato determinístico.

3.2 Tipo 2 — Linguagens Livres de Contexto e o Parser

As linguagens livres de contexto são reconhecidas por Autômatos de Pilha (PDA). A estrutura gramatical de linguagens de programação — como a regra de que uma atribuição é formada por um identificador, seguido de '=', seguido de uma expressão — é tipicamente livre de contexto. Isso significa que a validade de uma construção pode ser verificada sem precisar saber o que foi definido antes.

Na MiniLang, o analisador sintático implementa um PDA com pilha explícita. A gramática formal utilizada é:

```
PROGRAM → STMT
STMT    → ID = EXPR | print(ID)
EXPR    → VALOR RESTO
RESTO   → OP VALOR | ε
VALOR   → ID | NUM
```

A pilha registra cada operação PUSH e POP em um histórico, permitindo visualizar o funcionamento do PDA durante a análise de cada instrução.

3.3 Tipo 1 — Linguagens Sensíveis ao Contexto e o Semântico

As linguagens sensíveis ao contexto são reconhecidas por Autômatos Linearmente Limitados. Nesse nível, o significado de uma instrução depende do contexto em que ela aparece. Na análise semântica da MiniLang, isso se manifesta claramente: a instrução 'y = x + 5' só é válida se 'x' tiver sido definida anteriormente. O mesmo identificador 'x' pode ser válido ou inválido dependendo do contexto (das linhas anteriores do programa).

A tabela de símbolos implementada no analisador semântico mantém o estado entre as linhas do programa, registrando quais variáveis já foram definidas. Essa memória contextual é o que caracteriza uma linguagem sensível ao contexto.

3.4 Tipo 0 — Máquina de Turing e a Execução

A Máquina de Turing, proposta por Alan Turing em 1936, é o modelo computacional mais poderoso da hierarquia. Ela é capaz de reconhecer qualquer linguagem recursivamente enumerável e serve como modelo teórico para os computadores modernos. Na MiniLang, o executor simula uma Máquina de Turing simplificada: processa instruções sequencialmente, mantém uma memória (a fita) representada por um dicionário, e usa um acumulador como registrador interno para os cálculos.

4. Arquitetura do Compilador

O compilador foi dividido em módulos independentes organizados em subpastas, cada um responsável por uma etapa do processo de compilação:

```
A3-COMPILADORES/  
├── lexer/  
│   ├── afd.py                # AFD – estados e tabela de transição  
│   └── analisador_lexico.py  # Geração de tokens  
├── parser/  
│   ├── pilha.py              # Pilha com histórico de operações  
│   └── analisador_sintatico.py # Validação da gramática via PDA  
├── semantico/  
│   └── analisador_semantico.py # Tabela de símbolos e verificação CSL  
├── gerador/  
│   └── gerador.py            # Geração de pseudo-assembly  
├── executor/  
│   └── executor.py          # Execução via Máquina de Turing  
├── compilador.py            # Integração de todas as etapas  
├── main.py                  # Ponto de entrada  
└── programa.ml              # Arquivo de código fonte MiniLang
```

5. Etapas do Compilador

5.1 Análise Léxica — AFD

Arquivo: `lexer/afd.py` e `lexer/analizador_lexico.py`

O analisador léxico transforma o código fonte em uma sequência de tokens, percorrendo o código caractere por caractere. A 5-tupla formal do AFD foi implementada da seguinte forma:

- Q (estados): INICIO, LENDO_ID, LENDO_NUM, LENDO_OPERADOR, LENDO_PAREN, FIM, ERRO
- Σ (alfabeto): categorias de caracteres — letra, dígito, operador, parêntese, espaço
- δ (função de transição): dicionário `tabela_de_transicao` em `afd.py`
- q_0 (estado inicial): INICIO
- F (estados de aceitação): FIM

Exemplo de tokens gerados para "`y = x + 5`":

```
[('ID', 'y'), ('OP', '='), ('ID', 'x'), ('OP', '+'), ('NUM', '5')]
```

5.2 Análise Sintática — PDA

Arquivo: `parser/pilha.py` e `parser/analizador_sintatico.py`

O analisador sintático valida se a sequência de tokens obedece à gramática da MiniLang usando um PDA com pilha explícita. A pilha registra cada operação em um histórico, tornando visível o funcionamento do autômato. Exemplo do histórico para `"print(x)"`:

PUSH PRINT	→ Pilha: ['PRINT']
POP PRINT	→ Pilha: []
PUSH)	→ Pilha: [')']
PUSH ID	→ Pilha: [')', 'ID']
PUSH (	→ Pilha: [')', 'ID', '(']
PUSH print	→ Pilha: [')', 'ID', '(', 'print']
POP print	→ Pilha: [')', 'ID', '(']
POP (	→ Pilha: [')', 'ID']
POP ID	→ Pilha: [')']
POP)	→ Pilha: []

5.3 Análise Semântica — CSL

Arquivo: `semantico/analizador_semantico.py`

O analisador semântico verifica o significado do código, garantindo que variáveis sejam usadas apenas após serem definidas. Uma tabela de símbolos é mantida e atualizada a cada linha processada. Exemplo de erro detectado:

```
y = x + 5    ← Erro semântico: variável 'x' não foi definida
```

5.4 Geração de Código Intermediário

Arquivo: `gerador/gerador.py`

O gerador converte os tokens em pseudo-assembly. Exemplo de código gerado para "`y = x + 5`":

```
LOAD x
ADD 5
STORE y
```

5.5 Execução — Máquina de Turing

Arquivo: `executor/executor.py`

O executor processa o pseudo-assembly sequencialmente. Os elementos da Máquina de Turing foram mapeados da seguinte forma:

- Fita (memória): dicionário Python que armazena variáveis e seus valores
- Acumulador: registrador temporário para cálculos intermediários
- Cabeçote: índice da instrução sendo processada
- Transições: cada instrução representa uma mudança de estado da máquina

6. Como a Teoria da Computação foi Aplicada

A tabela a seguir resume onde cada conceito teórico da Hierarquia de Chomsky foi aplicado no projeto:

Etapa	Conceito Teórico	Onde foi aplicado
Análise Léxica	AFD	afd.py — tabela de transição implementada como dicionário Python, percorre o código caractere por caractere transitando entre estados explícitos.
Análise Sintática	PDA	pilha.py + analisador_sintatico.py — pilha explícita com histórico de operações PUSH/POP que valida a gramática da MiniLang.
Análise Semântica	CSL	analisador_semantico.py — tabela de símbolos que verifica se variáveis foram definidas antes de serem usadas (sensível ao contexto anterior).
Geração de Código	Código Intermediário	gerador.py — converte tokens em pseudo-assembly (LOAD, STORE, ADD, SUB, PRINT) como ponte entre análise e execução.
Execução	Máquina de Turing	executor.py — processa instruções sequencialmente usando memória (fita) e acumulador (registrador), simulando uma MT simplificada.

7. Dificuldades e Decisões de Projeto

7.1 Representação do AFD como Dicionário

Uma das primeiras decisões do projeto foi a forma de implementar o Autômato Finito Determinístico no analisador léxico. A abordagem mais simples seria usar uma cadeia de condicionais (if/elif), mas essa solução não tornaria explícita a estrutura do autômato. Optou-se por representar a tabela de transição como um dicionário Python aninhado, onde cada chave é um estado e cada valor é outro dicionário mapeando categorias de caracteres para o próximo estado. Essa decisão tornou o código mais fiel à definição formal de um AFD e facilita a leitura e explicação durante a apresentação.

7.2 Pilha Explícita com Histórico

O analisador sintático poderia ter sido implementado como um parser recursivo descendente, que é mais simples de codificar. No entanto, como o objetivo do trabalho é demonstrar o funcionamento de um PDA, optou-se por implementar uma pilha explícita com registro de histórico. Cada operação de empilhamento e desempilhamento é registrada junto com o estado atual da pilha, permitindo visualizar exatamente como o autômato processa cada instrução.

7.3 Separação entre Verificação e Execução

Durante o desenvolvimento, surgiu a dúvida sobre onde realizar os cálculos das expressões: no analisador semântico ou no executor. A decisão foi manter o semântico apenas como verificador — ele confirma se as variáveis existem, mas não calcula valores. Os cálculos são realizados pelo executor ao processar o pseudo-assembly. Essa separação torna cada módulo mais coeso e fiel à sua responsabilidade teórica: o semântico verifica contexto (CSL), e o executor computa (Máquina de Turing).

7.4 Processamento Linha por Linha

A MiniLang processa o código linha por linha, passando cada linha pelas etapas de léxico, sintático e semântico antes de gerar o código intermediário. O código intermediário de todas as linhas é acumulado e enviado ao executor apenas no final. Essa abordagem simplifica o compilador e permite que a tabela de símbolos do semântico persista entre as linhas, viabilizando referências a variáveis definidas anteriormente como em $y = x + 5$.

7.5 Tratamento de Erros

Cada etapa do compilador detecta e reporta erros de forma independente. Um erro léxico (caractere inválido) é detectado pelo AFD. Um erro sintático (estrutura inválida) é detectado pelo parser. Um erro semântico (variável não definida) é detectado pelo analisador semântico. O compilador interrompe o processamento na primeira linha com erro e exibe uma mensagem descritiva, facilitando a identificação e correção do problema pelo usuário.

8. Exemplo Completo de Execução

O programa abaixo demonstra o funcionamento completo do compilador com o exemplo básico proposto:

Arquivo programa.ml:

```
x = 10
y = x + 5
print(y)
```

Código intermediário gerado:

```
LOAD 10
STORE x
LOAD x
ADD 5
STORE y
PRINT y
```

Execução (Máquina de Turing):

```
LOAD 10    → acumulador = 10
STORE x    → memoria[x] = 10
LOAD x     → acumulador = 10
```

```
ADD 5      → acumulador = 15  
STORE y    → memoria[y] = 15  
PRINT y    → imprime 15
```

Saída do programa:

15

9. Conclusão

O desenvolvimento do compilador MiniLang demonstrou na prática como os conceitos fundamentais da Teoria da Computação se aplicam diretamente na construção de sistemas reais. A Hierarquia de Chomsky, que muitas vezes parece abstrata no contexto teórico, tornou-se concreta ao ser mapeada para as etapas do compilador: linguagens regulares no léxico, linguagens livres de contexto no parser, linguagens sensíveis ao contexto no semântico e a Máquina de Turing na execução.

A decisão de separar cada etapa em um módulo independente, com comentários explicativos e exemplos embutidos no código, tornou o projeto mais didático e facilitou a compreensão de cada conceito isoladamente. Cada arquivo pode ser executado e testado de forma independente, demonstrando sua etapa específica do processo de compilação.

O resultado final é um compilador funcional capaz de processar programas escritos em MiniLang, detectar erros léxicos, sintáticos e semânticos com mensagens descritivas, gerar código intermediário em pseudo-assembly e executar esse código produzindo a saída correta — tudo isso com a fundamentação teórica explícita em cada módulo do projeto.